

Lesson 9, Part 2 Activity: Estimate One Slice of Your Project

CS 5388 / CS 4390 Software Project Management | Team activity, complete together

Team: ____ Team 6 – The Gentlemen ____ **Project:** _Universal Automotive Management App____

What this activity is for

Your Sprint 2 business case has to answer a question every sponsor asks: how much work would this take, and what would it cost? You are not going to build the system in this course. You are doing what a project manager does before a project is approved: estimating the work so a decision can be made.

Estimating a whole project at once hides mistakes. So you will start small. Pick one or two major parts of your project, called a slice, and estimate only that slice. A small slice lets you count everything, check every number, and see clearly where your assumptions matter.

You will estimate the same slice twice, using two methods that start from different information. Then you will compare the two answers, explain why they differ, and present a range with your assumptions written down. That range becomes the Estimation Appendix on your Sprint 2 portal page.

What to bring

1. Your Release 1 capability list (Step 1 of the Part 1 starter).
2. Your backlog from the Part 1 activity: user stories, MoSCoW categories, and planning poker point sizes.
3. Your similar-products research from Part 1 (small and big products, with timelines and team sizes).
4. The Part 2 slides, especially Slides 13 to 17 (sizing), 19 to 24 (team size), 25 to 27 (story points), and 31 to 33 (reconciling).

How the activity fits together

Step	What you do	What you end up with
1	Pick your slice	One or two major parts of your project, with their stories, screens, and data listed
2	Choose two estimation methods	Two methods that use different starting information
3	Estimate the slice with each method	Two effort estimates, both in team hours
4	Check the estimates for team size	Confidence that coordination costs are included
5	Compare the two estimates	A written explanation of why they differ
6	Present the result	A dated range, in hours, dollars, and weeks, ready for the business case

Key terms

Term	What it means in this activity
Slice	One or two major parts of your project that a user would recognize, such as “buy a permit.” You estimate only this.
Size	How much software the slice contains, measured in function points, use-case points, or story points.
Effort	Total team hours of work. If 5 people each work 10 hours, that is 50 hours of effort.
Productivity rate	Hours of effort per unit of size, such as 8 hours per function point. The rate drives the whole estimate.
Assumption	Any number you chose rather than measured. Label it, so a reader knows it could change.
Velocity	Story points a team completes per sprint. You have no sprint history yet, so yours is an assumption.
Loaded rate	The full hourly cost of one person: salary plus benefits, overhead, and tools.
Range	A low and a high estimate. Honest estimates are ranges, not single numbers.
O, M, P	Optimistic, most likely, and pessimistic estimates, used in the PERT formula $(O + 4M + P) \div 6$.

About the examples

Every italic example uses the same slice from the fictional Campus Parking System: “Buy a semester permit.” Following one slice through every method shows how the pieces connect. Your own numbers come from your project.

Step 1. Pick your slice

A good slice is a part of your project that a user would recognize and that is useful on its own. It usually covers 3 to 8 user stories from your backlog.

How to choose

1. Start with a Must-have capability that is central to your project. If it did not exist, the project would not make sense.
2. If you pick a second part, make it different in kind, such as one part users interact with and one part that produces reports or connects to another system.
3. Do not pick the easiest part. The slice should include some real complexity, such as payments, another system, or stored data.
4. Write down what is out of the slice, so your estimate has a clear boundary.

Question	Our slice
Slice name (one or two major parts)	Maintenance Tracking and Reminders
Why we chose it	This is the main use case of our project, we want to give users a place to track their maintenance and get reminders about them.
Screens or forms where users enter data	Login screen, home page, Maintenance login page,
Reports, results, or lookups users see	Complete maintenance history report, notifications of maintenance reminders
Data the slice stores	Vehicle VIN #, mileage, make, model, year, service type, service date
Other systems it connects to	Push notification service, GPS
Out of the slice (not included)	Maintenance Cost, Shop Recommendation, AI-powered assistance

Stories in the slice

User story	MoSCoW	Points
User creates their account	Must	2
User registers a vehicle	Must	3
Users logs a maintenance event	Must	5
Users receive reminders for their upcoming service	Must	8
User views their full-service history for their vehicle	Should	3
User enables GPS tracking from their device, so mileage automatically updates.	Could	8
Total points		29

Example slice, “Buy a semester permit”: a student registers a vehicle (3 points), chooses a lot and permit type (5), pays online (8), receives a digital permit by email (3), and looks up the permit later (2), for 21 points. It stores permit and vehicle records and connects to the student registry and a payment service. Citations and appeals are out of the slice.

Step 2. Choose two methods

Each method starts from different information. Choose two that do not share the same starting point, so that if they agree, the agreement means something (Part 2, Slide 31).

Method	It starts from	A good choice if	Watch out for
Analogous	Similar products and how long they took	You found at least one product close to yours	Copying a big company's timeline without adjusting
Function points	Counts of what users enter, see, and store	Your Step 1 table lists screens, results, and data	A productivity rate with no source
Use-case points	The actors and use cases in your slice	You can list who uses the slice and for what	Default adjustment factors used without saying so
Story points to cost	Your planning poker points and a velocity	Every story in the slice has points	Velocity is a guess until you run sprints

Suggested pairs

1. Function points plus analogous: one starts from your own design, the other from the market. The examples below use this pair.
2. Story points plus analogous: one starts from your backlog, the other from the market.
3. Function points plus story points: one starts from system features, the other from team judgment of story size.

Our two methods: 1. _____ **Story points** _____ 2. _____ **Analogous** _____

Why these two use different starting information: Story points start from our own backlog and the points we assigned, while analogous starts from similar products already on the market.

Step 3. Estimate the slice

Complete only the two sections (3A to 3D) that match your methods. Whatever method you use, finish with effort in team hours for your slice, so the two estimates can be compared in Step 5. Where you must choose a number, write it down and label it an assumption.

3A. Function points (Part 2, Slides 13 to 15)

What it measures: the functionality users get from the slice, counted in five categories. Your Step 1 table already lists most of what you need.

Category	What to count	Example from the permit slice
External Inputs (EI)	Forms where data enters the system	Register vehicle, purchase permit, update vehicle
External Outputs (EO)	Results the system calculates and sends	Permit confirmation and receipt
External Inquiries (EQ)	Simple lookups of stored data	Look up my permit, list lots with permits available
Internal Logical Files (ILF)	Groups of data the slice stores	Permit records, vehicle records
External Interface Files (EIF)	Data read from another system	Student registry, payment service

How to do it

1. Use your Step 1 table. Count each distinct input, output, lookup, and data group in the slice once.
2. Multiply each count by its weight and add the results. That total is your slice size in function points.
3. Choose a productivity rate in hours per function point. You have no history, so label it an assumption and test a low and a high value (Slide 15 shows 6 to 12 hours).
4. Multiply size by each rate to get a low and high effort in hours.

Category	Count	Weight	Points
External Inputs (EI)		4	
External Outputs (EO)		5	
External Inquiries (EQ)		4	
Internal Logical Files (ILF)		10	
External Interface Files (EIF)		7	
Total function points			

Rate: **low** _____ and **high** _____ hours per FP Assumption or source:

Effort: _____ FP × _____ h = _____ hours (low) _____ FP × _____ h = _____ hours (high)

Example: 3 inputs (12), 1 output (5), 2 inquiries (8), 2 data groups (20), and 2 external systems (14) give 59 function points. At 6 to 12 hours per function point (assumed), effort is 354 to 708 hours; at 8 hours it is 472.

3B. Use-case points (Part 2, Slides 16 and 17)

What it measures: size based on who uses the slice (actors) and what they do with it (use cases). A use case is a goal a user completes, such as “purchase a permit.” Its steps are the back-and-forth exchanges between the user and the system.

How to do it

1. List your actors. A person using a screen is complex (3). Another system connected through a protocol is average (2). Another system connected through an API is simple (1). Add the weights to get UAW.
2. List the use cases in your slice and count the steps in each. 1 to 3 steps is simple (5), 4 to 7 is average (10), 8 or more is complex (15). Add the weights to get UUCW.
3. Add UAW and UUCW to get unadjusted use-case points (UUCP).
4. The technical factor (TCF) and environment factor (ECF) adjust for difficulty and team experience. If you do not score the full factor lists, use 1.0 for each and write that down as an assumption.
5. Multiply to get UCP, then multiply by a rate. Karner’s starting rate is 20 hours per UCP; label it an assumption.

Item	Calculation	Value
Actor weight (UAW)	Sum of actor weights	
Use case weight (UUCW)	Sum of use case weights	
Unadjusted (UUCP)	$UAW + UUCW$	
Technical factor (TCF)	1.0 unless you scored it	
Environment factor (ECF)	1.0 unless you scored it	
Use-case points (UCP)	$UUCP \times TCF \times ECF$	
Rate (hours per UCP)	20 unless you have a source	
Effort (hours)	$UCP \times \text{rate}$	

Example: the student is a complex actor (3); the registry and payment service connect by API (1 each), so UAW = 5. Register vehicle is simple (5), purchase permit has 8 or more steps (15), and look up permit is simple (5), so UUCW = 25. UCP = 30 with factors of 1.0 (assumed). At 20 hours each, effort is 600 hours.

3C. Story points to cost (Part 2, Slides 25 to 27)

What it measures: effort based on your team's own sizing of the stories in the slice. Story points are relative, so you convert them using a velocity (points per sprint). Because you have not run any sprints, velocity is an assumption. Use a low, middle, and high value.

How to do it

1. Use the total points from your Step 1 story table.
2. Choose a low, middle, and high velocity in points per two-week sprint. Label them assumptions.
3. Divide the slice points by each velocity. Keep the decimals; a small slice may take less than one sprint.
4. Multiply by team hours per sprint (team size $\times$ hours per person per sprint) to get effort in hours.

Input	Value	Note
Slice points	29 (Assumption)	From your Step 1 story table
Team size	4	Five or six at most
Hours per person per sprint	80 (Assumption)	State your assumption
Velocity: low / middle / high	15 / 20 / 25 (Assumptions)	Assumptions until you have sprint data
Sprints needed (high to low velocity)	1.2 / 1.5 / 1.9	Slice points $\div$ velocity
Effort in hours (low to high)	371 / 464 / 619	Sprints $\times$ team size $\times$ hours per person per sprint

Example: 21 points at 30, 20, or 15 points per sprint takes 0.7, 1.05, or 1.4 sprints. With 5 people working 80 hours each per sprint (400 team hours), effort is 280 to 560 hours.

3D. Analogous (Part 2, Slide 10)

What it measures: effort based on how long a similar real product took. You turn the product's timeline into person-months, scale it down to the size of your slice, then convert it to hours.

How to do it

1. Pick the similar product closest to your project from your Part 1 research, and note its team size and time to first release.
2. Multiply team size by months to get person-months. This is the effort the comparable product took.
3. Estimate what fraction of their first release your slice matches. If your slice is about a tenth of what they shipped, multiply by 0.1. Give a low and high fraction and explain them.
4. Adjust for team size. A much larger team spent more of its time coordinating, so its effort per feature is higher than yours will be.
5. Convert to hours. State the hours per person-month you assume (for example, 160).

Question	Answer
Comparable product, team size, and time to first release	AUTOsist, 1 person, ~12 months
Person-months (team size $\times$ months)	12 person-months
Fraction of their release our slice matches (low to high), and why	20%-40% Our slice covers AUTOsist's core functions (register cars, log maintenance, view history, reminders) but excludes a few extra things like cross-platform functionality and UI polish.
Team size adjustment and why	+15% We have a team of 4 which is larger than AUTOsist's 1. Since we will spend more time coordinating/meeting, we add a margin.
Adjusted effort in hours (low to high)	$12 \times 0.2 \text{ to } 0.4 = 2.4 \text{ to } 4.8 \text{ person-months} \times 1.15 = 2.8 \text{ to } 5.5 \text{ person-months} \times 160 \text{ hrs/person-month} = \sim 448 \text{ to } 880 \text{ hours}$
Confidence (low, medium, high) and why	Medium, the fraction percent is based only off our judgement since we only have AUTOsist's overall timeline, not its actual feature-by-feature breakdown.

Example: ParkEasy (fictional) took 4 people about 9 months, or 36 person-months, to launch a map of lots and pay-by-phone in one city. Buying a permit is roughly 8 to 12 percent of that release. $36 \times 0.08 \text{ to } 0.12 = 2.9 \text{ to } 4.3$ person-months, or about 460 to 690 hours at 160 hours per person-month (about 576 hours at 10 percent).

Before you move on: put both estimates in hours

Step 5 compares your two estimates directly, so both must be in team hours for the same slice. If an estimate came out in months, sprints, or dollars, convert it now and show the conversion.

Step 4. Check your estimates for team size (Part 2, Slides 19 to 24)

Larger teams spend more of their time coordinating: meetings, handoffs, and fitting work back together. Estimators tend to underweight this cost, so check each estimate against the list below. Anything marked No needs a fix or a written explanation.

Check	Yes / No	Why it matters
Our team is five or six people at most.	Yes	Beyond six, split into two teams.
We counted our communication paths: $n(n - 1) \div 2 =$ __6__	Yes	Five people share 10 paths; ten share 45.
We added explicit work for integrating the pieces of the slice.	Yes	Breaking work into pieces hides the cost of putting it back together.
We did not assume that more people means proportionally more output.	Yes	Coordination grows faster than headcount.
We estimated for the team we have.	Yes	Not for a larger team we wish we had.
We adjusted, not copied, timelines from much larger teams.	Yes	Their effort includes their coordination costs.

Step 5. Compare your two estimates (Part 2, Slides 31 and 32)

Two estimates almost never match. The gap is useful: it points to the assumption you are least sure about. Your job is to find that assumption and name it.

	Method 1	Method 2
Method	Story points	Analogous
Estimate in hours (low to high)	371 / 464 / 619	448 to 880 hours
Middle value in hours	464	664
Input that drives it most	Velocity assumption	The fraction of AUTOSist's release our slice matches
Is that input measured or assumed?	Assumed since no sprint data exists yet	Assumed, we judge it as a team

Gap: larger middle value ÷ smaller middle value = 1.4

If one estimate is more than about twice the other, recheck your counts and conversions before explaining the gap.

Questions to answer

1. Do your two methods depend on the same assumption? If so, their agreement proves little.
2. Which single input moves the answer most? Try changing it and see.
3. What information would narrow the gap, and when could you get it?

Our explanation of the gap (one paragraph)
Story points gave a middle value of 464 hours, and the analogous method gave 664 hours, a gap of about 1.4. The two are decently close, and they do start from different information, so the agreement is meaningful. The largest uncertainty is the fraction of AUTOSist's release that matches our slice, which is our judgment based on feature scope. The second uncertainty is our chosen velocity, which is unvalidated since we haven't actually run any sprints.

Example: function points gave a middle value of 472 hours and the analogous method gave 576 hours, a gap of about 1.2. The two are fairly close, and they start from different information, so the agreement is meaningful. The largest uncertainty is the productivity rate, which is our assumption, and the fraction of ParkEasy's release that our slice represents.

Step 6. Present the estimate (Part 2, Slide 33)

This is the table your sponsor reads. It gives a range instead of a single number, shows what the range depends on, and is dated because estimates change as you learn more. It becomes the Estimation Appendix on your Sprint 2 portal page.

How to fill it in

1. Range: the lowest and highest hours across both methods.
2. Expected value: set O to your low end and P to your high end. Choose M, your most likely value, somewhere between them and explain why. Then calculate $(O + 4M + P) \div 6$.
3. Cost: multiply hours by a loaded hourly rate. State the rate and where it came from, such as published wage data plus an overhead percentage you choose.
4. Calendar time: divide hours by (team size $\times$ productive hours per person per week). State the productive hours you assume.

Field	Our estimate
Slice estimated	Maintenance reminders and tracking
Range in hours (low to high)	371-880 Hours
Expected hours: $(O + 4M + P) \div 6$, and why we chose M	O = 371, M= 564, P = 880 Expected: $(371 + 4(564) + 880) / 6 \dots = 584.5$ Hours. (564 was chosen since it is the midpoint between the estimates of our 2 methods.
Cost range and loaded rate used	About \$25,970- \$61,600 with a rate of around \$70/hour
Calendar estimate in weeks, and hours per week assumed	Around 3.7 weeks are expected, with a range of 2.3-5.5 weeks. We are assuming 40 hours of productivity per team member per week. Since there are 4 of us we are looking at about 160 hours of productivity of the team per week.
Methods used	The Story point estimation and analogous estimation
Key assumptions	15-25 points per sprint, 80 productive hours per 2 week window (1 sprint). Also, a 4 person team.
Team size assumed	4-person team.
What would change this estimate	The pace at how we work in the sprints, difficulty on being able to integrate our ideas into the software, are things that could change the estimate.
Date of this estimate	September 29,2026

“Maintenance reminders and tracking” Slice: range 371 to 880 hours. With M = 564, expected hours $(371 + 2,256 + 880) / 6 = 584.5$ Hours. At an assumed loaded rate of \$60 per hour, cost is about \$22,260 to \$52,800. With 4 people at an assumed 40 productive hours a week (160 team Hours), the expected 584.5 hours are about 3.7 weeks or so.

Example, “Buy a semester permit” slice: range 354 to 708 hours. With M = 520, expected hours = $(354 + 2,080 + 708) \div 6 \approx 524$. At an assumed loaded rate of \$60 per hour, cost is about \$21,000 to \$42,500. With 5 people at an assumed 30 productive hours a week (150 team hours), the expected 524 hours is about 3.5 weeks.

Going further (optional)
Once your slice estimate is solid, you can use it to make a rough estimate of the whole project. If your slice holds about one fifth of your backlog points, the whole project is roughly five times the slice. Treat that number as a first rough guess, and repeat this activity on more slices as you learn more.

Before you submit

1. Your slice is named, with its stories, screens, data, and boundary written down.
2. Both estimates are for the same slice and in hours, with the conversions shown.
3. Every assumed number is labeled as an assumption.
4. Every sourced number has a source you can link.
5. The team-size check is complete, and any No has a fix or explanation.
6. The final estimate is a dated range, not a single number.
7. The individual estimation memo is your own work, written without AI.